2.16 Дженерики

Введение

Дженерики в Go — это возможность создавать функции и типы, которые могут работать с любыми типами данных, соответствующими определенным ограничениям.

Эта функция, добавленная в Go 1.18 в марте 2022 года, является одним из самых значительных обновлений языка (Go 1.18 Release Notes).

Зачем нужны дженерики?

До дженериков разработчикам приходилось писать несколько версий одной функции для разных типов данных.

Функция SumInts

```
func SumInts(s []int) int {
    sum := 0
    for _, v := range s {
        sum += v
    }
    return sum
}
```

Функция SumFloats

```
func SumFloats(s []float64) float64 {
    sum := 0.0
    for _, v := range s { sum +=
        v
    }
    return sum
}
```

Generic функция Sum

```
package main

import "fmt"

func Sum[T int | float64](s []T) T {
    var sum T
    for _, v := range s {
        sum += v
    }
    return sum
}

func main() {
    ints := []int{1, 2, 3}
    floats := []float64{1.5, 2.5, 3.5}
    fmt.Println(Sum(ints))
    fmt.Println(Sum(floats))
}
```

Основы синтаксиса

Дженерики позволяют определять функции и типы с параметрами типов, которые указываются в квадратных скобках. Ограничения типов задаются через интерфейсы, определяющие допустимые типы.

Generic функции

Generic функция использует параметры типов перед именем функции.

```
package main

func max[T int|float64](a, b T) T {
    if a > b {
        return a
    }
    return b
}

func main() {
    println(max(3, 4))
    println(max(3.5, 4.5))
    // println(max(uint(3), uint(4)))
}
```

- `T` — параметр типа.
- `int|float64` — ограничение, требующее, чтобы тип был либо `int` , либо `float64`

Generic типы

Типы, такие как структуры, также могут быть generic.

```
type ListNode[T any] struct {
    Value T
    Next *ListNode[T]
}

func (n *ListNode[T]) Add(value T) *ListNode[T] {
    newNode := &ListNode[T]{Value: value}
    newNode.Next = n.Next
    n.Next = newNode
    return newNode
}
```

`T any` означает, что `T` может быть любым типом.

Этот тип позволяет создавать списки для любых данных, от чисел до пользовательских структур.

Ограничения типов

Ограничения задаются через интерфейсы. Например, можно создать собственное ограничение для числовых типов:

```
type Number interface {
    int64 | float64
}

func SumNumbers[K comparable, V Number](m map[K]V) V {
    var sum V
    for _, v := range m {
        sum += v
    }
    return sum
}
```

Инференс типов

Go автоматически выводит типы параметров, если это возможно, что упрощает вызов generic функций (Tutorial: Getting started with generics):

```
package main

  func SumIntsOrFloats[K comparable, V int64 | float64](m map[K]V) V {
      var sum V
      for _, v := range m {
          sum += v
      }
      return sum
  }

  func main() {
      ints := map[string]int64{"a": 1, "b": 2}
      floats := map[string]float64{"a": 1.5, "b": 2.5}

      println(SumIntsOrFloats(ints))
      println(SumIntsOrFloats[string, float64](floats))
  }
```

Компилятор определяет, что `V` — это `int64` для `ints` и `float64` для `floats`.

Примеры

Функция max

```
package main

import "golang.org/x/exp/constraints"

func max[T
    constraints.Ordered](a, b T) T
    { if a > b {
        return a
    }
    return b
}
```

constraints.Ordered — ограничение, требующее, чтобы тип поддерживал операторы сравнения (<, >), например, int, float64, string (Constraints Package).

MERION

Реверс слайса

Generic функция для реверса слайса работает с любым типом элементов:

```
func Reverse[T any](s []T) {
    for i, j := 0, len(s)-1; i < j; i, j = i+1, j-1 {
        s[i], s[j] = s[j], s[i]
    }
}

func main() {
    ints := []int{1, 2, 3, 4}
    Reverse(ints)
    fmt.Println(ints)

    strings := []string{"a", "b", "c"}
    Reverse(strings)
    fmt.Println(strings)
}
```

Когда использовать дженерики

- **Контейнерные типы**: Слайсы, мапы, каналы, где тип элемента не влияет на логику.
- **Общие структуры данных**: Связанные списки, бинарные деревья.
- **Устранение дублирования**: Когда одинаковый код повторяется для разных типов.

Когда не использовать дженерики

- **Если интерфейсы подходят**: Например, `io.Reader` для чтения данных.
- **Разные реализации методов**: Используйте интерфейсы, если поведение зависит от типа

Распространенные ошибки

- **Неправильные ограничения**: Использование операций, не поддерживаемых типом, вызывает ошибки компиляции.
- **Злоупотребление дженериками**: Не все функции должны быть generic.
- **Проблемы с инференсом типов**: Иногда нужно явно указывать типы, если компилятор не может их вывести.

Итог

Дженерики в Go позволяют писать универсальный и безопасный код, уменьшая дублирование и повышая гибкость.